

系统开发工具基础 · 实验报告

第 4 周：代码质量 · 元编程 · 大杂烩 · 课堂综合测试

姓 名	徐宜安	学 号	25060012033
授课教师	周小伟	上课日期	2026 年 9 月 14 日
Git 仓库	GitHub 课程仓库	报告日期	2026 年 9 月 9 日

1 课上练习

1.1 q13 · 建立可执行的本地质量门禁（代码质量）

题目要求

复用 q10 完成的 greetlab 包，在 q13 中建立轻量本地质量检查：

- 在 `pyproject.toml` 中加入 Ruff 和 Pytest 的最小配置；
- 保留正常姓名与空白姓名两个带有效断言的测试；
- 运行 `ruff format`、`ruff check` 和 `pytest`，修复全部问题且不全局忽略规则；
- 编写 `check.sh` 依次执行三项检查，并保证返回 0。

实验过程：首先把 q10 复制为 q13，并以可编辑方式安装当前项目，同时确认 Ruff 和 Pytest 已经安装。随后在 `pyproject.toml` 中加入格式、检查规则和测试路径配置，其中 Ruff 选择 E、F、W 三组规则，Pytest 明确从 `tests` 收集测试，并把 `src` 加入 Python 搜索路径。本题新增的核心配置如下。

```
1 [tool.ruff]
2 line-length = 120
3
4 [tool.ruff.lint]
5 select = ["E", "F", "W"]
6
7 [tool.pytest.ini_options]
8 testpaths = ["tests"]
9 pythonpath = ["src"]
```

代码 1 本题新增的 Ruff 与 Pytest 配置

第一次执行 `ruff format --check` 时，终端指出 `src/greetlab/cli.py` 和 `tests/test_cli.py` 两个文件需要重新格式化，具体差异是导入语句与函数之间缺少规范空行。项目复制、依赖安装和首次检查结果见图 1。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上 (main)
• $ cp -r ../../第三周/课上/q10 q13;cd q13

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q13 (main)
• $ pip install -e . ruff pytest
Obtaining file:///D:/desktop/%E7%B3%BB%E7%BB%9F%E5%BC%80%E5%8F%91%E5%B7%A5%E5%85%B7%E5%9F%BA%E7%A1%80/%E7%AC%AC%E5%9B%9B%E5%8A/q13
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Requirement already satisfied: ruff in D:\Software\Lib\site-packages (0.16.5)
Requirement already satisfied: pytest in D:\Software\Lib\site-packages (9.1.1)
```

(a) 复制 q10 并安装当前项目、Ruff 与 Pytest

```
[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q13 (main)
• $ ruff format --check
unformatted: File would be reformatted
--> src\greetlab\cli.py:3:1
2 |
3 | +
4 | def main():
-----
9 |         raise SystemExit(2)
-     print(f"Hello, {a.name}!")
10 +     print(f"Hello, {a.name}!")
|

unformatted: File would be reformatted
--> tests\test_cli.py:5:1
4 |
5 | +
6 | def test_main_normal(monkeypatch, capsys):
-----
11 |
12 +
13 | def test_main_blank(monkeypatch):
|

2 files would be reformatted, 2 files already formatted
```

(b) 首次格式检查指出两个文件需要调整

图 1 q13 环境准备与首次 Ruff 格式检查

执行 `ruff format`，后两个文件被自动整理，再运行 `ruff check`，和 `pytest`，代码检查全部通过，两个测试也均通过。最后把三条命令写入 `check.sh`，其中 `set -euo pipefail` 保证任一检查失败时脚本立即以非零状态结束。

```
1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  ruff format --check .
5  ruff check .
6  pytest
```

代码 2 依次执行格式、代码和测试检查的本地门禁

单独执行三项检查的结果见图 2(a)。为脚本增加执行权限后运行 `./check.sh`，三项检查再次通过，`echo $?` 输出 0，见图 2(b)。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q13 (main)
• $ ruff format .
2 files reformatted, 2 files left unchanged

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q13 (main)
• $ ruff check .
All checks passed!

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q13 (main)
• $ pytest
===== test session starts =====
platform win32 -- Python 3.11.8, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\desktop\系统开发工具基础\第四周\课上\q13
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.12.0
collected 2 items

tests\test_cli.py .. [100%]

===== 2 passed in 0.03s =====
```

(a) 格式化后 Ruff 检查和两个 Pytest 测试全部通过

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q13 (main)
$ chmod +x check.sh
./check.sh
• echo $?
4 files already formatted
All checks passed!
===== test session starts =====
platform win32 -- Python 3.11.8, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\desktop\系统开发工具基础\第四周\课上\q13
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.12.0
collected 2 items

tests\test_cli.py .. [100%]

===== 2 passed in 0.02s =====
0
```

(b) 运行统一检查脚本并确认退出码为 0

图 2 q13 修复格式问题并运行本地质量门禁

实验结果： q13 已建立可重复执行的本地质量门禁。当前代码格式、Ruff 规则和两个功能测试全部通过，脚本正常结束并返回 0。

1.2 q14 · 让 Make 只重建真正受影响的产物（元编程：构建系统）

题目要求

根据给定的 data.csv、stats.py、build_report.py 和 report.md 编写 Makefile：

- 包含 all、stats.txt、report.txt 和 clean；
- 准确列出数据文件、脚本和中间产物依赖，并将 all、clean 声明为 .PHONY；
- 验证首次构建、无改动时不执行配方，以及更新 data.csv 后的依赖传播；
- 确认 clean 只删除生成物。

实验过程： q14 初始包含 Makefile 和题目给出的四个源文件。stats.txt 依赖 data.csv 与 stats.py；report.txt 除了依赖报告模板和生成脚本，还依赖前一步得到的 stats.txt。这样数据变化时会先重算统计值，再重建最终报告。Makefile 内容如下。

```
1 .PHONY: all clean
2
3 all: stats.txt report.txt
4
```

```
5 stats.txt: data.csv stats.py
6     python stats.py
7
8 report.txt: report.md stats.txt build_report.py
9     python build_report.py
10
11 clean:
12     rm -f stats.txt report.txt
```

代码 3 统计结果与文本报告的构建依赖

第一次运行 `make` 时, Make 依次执行两个 Python 脚本, 生成 `stats.txt` 和 `report.txt`, 过程见图 3。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q14 (main)
$ alias make='/d/Software/ucrt64/bin/mingw32-make.exe'
• make --version
GNU Make 4.4.1
Built for Windows32
Copyright (C) 1988-2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q14 (main)
• $ make
python stats.py
python build_report.py
```

图 3 q14 首次执行 Make 并运行两个生成脚本

统计值为 10, 最终报告包含标题和 Total: 10, 见图 4(a)。在没有修改任何文件的情况下再次执行 `make`, 终端提示 Nothing to be done for 'all', 没有重复运行脚本, 见图 4(b)。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q14 (main)
• $ cat stats.txt
10
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q14 (main)
• $ cat report.txt
# Data Report

Total: 10
```

(a) 统计文件与最终报告的内容

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q14 (main)
• $ make
mingw32-make: Nothing to be done for 'all'.
```

(b) 无改动时 Make 不重复执行配方

图 4 q14 构建结果与无改动时的增量检查

接着使用 `touch data.csv` 更新数据文件的修改时间。再次执行 `make` 时, `stats.py` 与 `build_report.py` 都被重新运行, 说明中间产物的依赖继续传递到最终报告。数据内

容没有改变，因此两个产物的内容仍为 10。最后执行 `make clean`，只删除 `stats.txt` 和 `report.txt`；`Makefile` 与四个源文件均被保留。相关过程见图 5。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q14 (main)
• $ touch data.csv

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q14 (main)
• $ make
python stats.py
python build_report.py

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q14 (main)
• $ cat stats.txt
10
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q14 (main)
• $ cat report.txt
# Data Report

Total: 10

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q14 (main)
• $ make clean
rm -f stats.txt report.txt
```

图 5 更新数据后重新构建，并用 `clean` 删除两个生成物

实验结果： Make 能根据依赖文件的修改时间决定是否执行配方。首次构建生成两个产物，无改动时不重复执行，数据文件更新后两个产物按依赖顺序重建，清理目标也没有删除源文件。

1.3 q15 · 把本地 API 数据转换为可读报告（API、jq 与 CLI）

题目要求

把题目给定的 JSON 保存为 `packages.json`，并完成以下操作：

- 在 q15 中运行本地 HTTP 服务，使用 `curl -fsS` 获取 JSON；
- 用 `jq` 筛选状态为 `active` 且下载量不少于 100 的记录；
- 按下载量降序、名称升序排列；
- 编写 `api_report.sh`，生成包含标题与三列表格的 `summary.md`。

实验过程： 首先在 q15 保存包含五条记录的 `packages.json`，在一个终端中运行 `python -m http.server 8000`，再从另一个终端使用 `curl -fsS` 请求 `http://127.0.0.1:8000/packages.json`。服务端记录显示请求返回 HTTP 200，客户端也成功取得并格式化显示全部 JSON，见图 6。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础 (main)
$ cd /d/desktop/系统开发工具基础/第四周/课上/q15
export PATH="/d/Software/ucrt64/bin:$PATH"

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q15 (main)
$ curl -fsS http://127.0.0.1:8000/packages.json | jq .
[
  {
    "name": "alpha",
    "status": "active",
    "downloads": 120,
    "version": "1.2.0"
  },
  {
    "name": "beta",
    "status": "inactive",
    "downloads": 900,
    "version": "2.0.0"
  },
  {
    "name": "gamma",
    "status": "active",
    "downloads": 450,
    "version": "1.5.1"
  },
  {
    "name": "delta",
    "status": "active",
    "downloads": 450,
    "version": "0.9.0"
  },
  {
    "name": "epsilon",
    "status": "active",
    "downloads": 80,
    "version": "3.1.0"
  }
]

Microsoft Windows [版本 10.0.26200.9168]
(c) Microsoft Corporation. 保留所有权利。

D:\desktop\系统开发工具基础>cd 第四周\课上\q15

D:\desktop\系统开发工具基础\第四周\课上\q15>python -
m http.server 8000
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
::ffff:127.0.0.1 - - [08/Sep/2026 05:49:04] "GET /pa
ckages.json HTTP/1.1" 200 -
[]
```

图 6 q15 启动本地 HTTP 服务并使用 curl 获取 JSON 数据

随后用 jq 保留 status 为 active 且 downloads 不少于 100 的记录，并用负下载量与名称组成排序键。筛选后剩下 delta、gamma、alpha 三项，下载量相同的 delta 与 gamma 按名称升序排列，结果见图 7。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q15 (main)
$ curl -fsS http://127.0.0.1:8000/packages.json |
jq '[.[] | select(.status == "active" and .downloads >= 100)] | sort_by([-.downloads, .name])'
[
  {
    "name": "delta",
    "status": "active",
    "downloads": 450,
    "version": "0.9.0"
  },
  {
    "name": "gamma",
    "status": "active",
    "downloads": 450,
    "version": "1.5.1"
  },
  {
    "name": "alpha",
    "status": "active",
    "downloads": 120,
    "version": "1.2.0"
  }
]
```

图 7 q15 使用 jq 按条件筛选并按两个关键字排序

最后把获取、筛选、排序和 Markdown 行生成写进脚本。脚本先输出标题与表头，再用 jq 的 -r 参数生成三行不带 JSON 引号的表格内容，核心代码如下。

```
1 {
2   echo "# Active Package Report"
```

```
3  echo
4  echo "| name | version | downloads |"
5  echo "|---|---|---:|"
6  curl -fsS "$url" | jq -r '
7      [.] | select(.status == "active" and .downloads >= 100)]
8      | sort_by([-.downloads, .name]) | .[]
9      | "| \(.name) | \(.version) | \(.downloads) |"
10 '
11 } > summary.md
```

代码 4 从本地 API 生成 Markdown 表格的核心代码

赋予脚本执行权限并运行后，生成的 `summary.md` 包含正确标题、列名和三条记录，最终内容见图 8。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q15 (main)
$ chmod +x api_report.sh
./api_report.sh
● cat summary.md
# Active Package Report

| name | version | downloads |
|---|---|---:|
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
```

图 8 q15 脚本生成的 Markdown 三列表格

实验结果：本地 HTTP 服务能够正常提供 JSON 数据。脚本最终保留下载量分别为 450、450 和 120 的三条有效记录，并按 delta、gamma、alpha 的顺序生成 Markdown 表格。

1.4 q16 · 修复并交付一个陌生的小型工具仓库（课堂综合测试）

题目要求

复用 q13 的项目完成一次综合检查：

- 临时把问候语实现改为固定输出 `Hello, name!`，运行质量门禁确认测试失败，再定位并修复；
- 编写包含 `check`、`build` 和 `clean` 的最小 Makefile；
- 运行 `make check` 和 `make build`，生成 wheel 并计算 SHA-256；
- 将最终改动提交为内容聚焦的 Git 提交。

实验过程：从 q13 复制源码、测试、项目配置和检查脚本到 q16，并继续使用课程根目录的 Git 仓库。为了验证测试是否真正约束了程序行为，先把问候语临时改成固定字符串 `Hello, name!`。此时格式和静态检查仍然通过，但正常姓名测试得到的实际输出与期望不符，Pytest 显示 1 failed, 1 passed，失败位置和差异见图 9。

```

xya@DESKTOP-URD5PIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q16 (main)
$ mkdir -p src
cp -r ../q13/src/greetlab src/
cp -r ../q13/tests .
cp ../q13/pyproject.toml ../q13/check.sh .
rm -rf src/greetlab/__pycache__ tests/__pycache__
chmod +x check.sh

xya@DESKTOP-URD5PIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q16 (main)
$ ./check.sh
3 files already formatted
All checks passed!
===== test session starts =====
platform win32 -- Python 3.11.8, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\desktop\系统开发工具基础\第四周\课上\q16
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.12.0
collected 2 items

tests\test_cli.py F. [100%]

===== FAILURES =====
_____ test_main_normal _____

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x0000021C0C215490>
capsys = <_pytest.capture.CaptureFixture object at 0x0000021C0C223610>

    def test_main_normal(monkeypatch, capsys):
        monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", "25060012033"])
        main()
        out, _ = capsys.readouterr()
        > assert out.strip() == "Hello, 25060012033!"
E       AssertionError: assert 'Hello, name!' == 'Hello, 25060012033!'
E
E       - Hello, 25060012033!
E       + Hello, name!

tests\test_cli.py:9: AssertionError
===== short test summary info =====
FAILED tests/test_cli.py::test_main_normal - AssertionError: assert 'Hello, name!' == 'Hello, 25060012033!'
===== 1 failed, 1 passed in 0.17s =====

```

图 9 q16 临时引入固定问候语后，正常姓名测试准确失败

根据失败断言，把固定字符串恢复为使用 `a.name` 的格式化输出。再次执行 `check.sh` 后两个测试均通过，退出码为 0，见图 10(a)。随后编写最小 Makefile：check 调用已有质量门禁，build 生成 wheel，clean 只清理构建目录、分发目录和包元数据。

```

1 .PHONY: check build clean
2
3 check:
4     ./check.sh
5
6 build:
7     python -m build --wheel
8
9 clean:
10    rm -rf build dist src/*.egg-info

```

代码 5 质量检查、wheel 构建与清理目标

运行 `make check` 后，Makefile 成功调用检查脚本，格式、代码和测试检查全部通过，结果见图 10(b)。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q16 (main)
• $ ./check.sh
echo $?
3 files already formatted
All checks passed!
===== test session starts =====
platform win32 -- Python 3.11.8, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\desktop\系统开发工具基础\第四周\课上\q16
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.12.0
collected 2 items

tests\test_cli.py .. [100%]

===== 2 passed in 0.02s =====
0
```

(a) 修复问候语后直接运行质量门禁

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q16 (main)
• $ alias make='d:/Software/ucrt64/bin/mingw32-make.exe'

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q16 (main)
• $ make check
./check.sh
3 files already formatted
All checks passed!
===== test session starts =====
platform win32 -- Python 3.11.8, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\desktop\系统开发工具基础\第四周\课上\q16
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.12.0
collected 2 items

tests\test_cli.py .. [100%]

===== 2 passed in 0.02s =====
```

(b) 通过 Makefile 的 check 目标运行同一套检查

图 10 q16 修复后的直接检查与 Make 检查

接下来执行 `make build`。构建工具创建隔离环境并打包项目，最终生成 `greetlab_25060012033-0.1.0-py3-none-any.whl`。由于构建输出较长，图 11(a) 连续保留开始与成功结束两部分。随后使用 `sha256sum dist/*.whl` 计算构建产物摘要，得到 `006eb8581c99b33a48182971d20e573109202bea41ba76b16f781da68fd18f6f`，见图 11(b)。

最后只暂存 Makefile、检查脚本、项目配置、源码和测试文件，提交信息为“完成 q16 综合质量检查与构建流程”。执行提交命令的记录见图 11(c)，Git 日志中对应提交为 `99480f6`。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q16 (main)
$ make build
python -m build --wheel
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
adding 'greetlab_25060012033-0.1.0.dist-info/entry_points.txt'
adding 'greetlab_25060012033-0.1.0.dist-info/top_level.txt'
adding 'greetlab_25060012033-0.1.0.dist-info/RECORD'
removing build\bdist.win-amd64\wheel
Successfully built greetlab_25060012033-0.1.0-py3-none-any.whl
```

(a) wheel 构建开始与成功结束记录

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课上/q16 (main)
$ sha256sum dist/*.whl
006eb8581c99b33a48182971d20e573109202bea41ba76b16f781da68fd18f6f *dist/greetlab_25060012033-0.1.0-py3-none-any.whl
```

(b) wheel 的 SHA-256 摘要

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础 (main)
$ git add \
  第四周/课上/q16/Makefile \
  第四周/课上/q16/check.sh \
  第四周/课上/q16/pyproject.toml \
  第四周/课上/q16/src/greetlab/__init__.py \
  第四周/课上/q16/src/greetlab/cli.py \
  第四周/课上/q16/tests/test_cli.py
warning: in the working copy of '第四周/课上/q16/src/greetlab/cli.py', LF will be replaced by CRLF the next time Git touches it

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础 (main)
$ git commit -m "完成 q16 综合质量检查与构建流程"
[main 99480f6] 完成 q16 综合质量检查与构建流程
6 files changed, 63 insertions(+)
create mode 100644 "\347\254\254\345\233\233\345\221\250\350\257\276\344\270\212\q16\Makefile"
create mode 100644 "\347\254\254\345\233\233\345\221\250\350\257\276\344\270\212\q16\check.sh"
create mode 100644 "\347\254\254\345\233\233\345\221\250\350\257\276\344\270\212\q16\pyproject.toml"
create mode 100644 "\347\254\254\345\233\233\345\221\250\350\257\276\344\270\212\q16\src\greetlab\__init__.py"
create mode 100644 "\347\254\254\345\233\233\345\221\250\350\257\276\344\270\212\q16\src\greetlab\cli.py"
create mode 100644 "\347\254\254\345\233\233\345\221\250\350\257\276\344\270\212\q16\tests\test_cli.py"
```

(c) 只暂存核心文件并执行 Git 提交

图 11 q16 wheel 构建、完整性摘要与最终提交

实验结果：测试能够发现固定问候语带来的行为错误，修复后本地质量门禁与 make check 均通过；make build 成功生成 wheel，SHA-256 摘要计算完成，核心代码和构建流程也已提交到 Git 仓库。

2 课后练习

参考资料记录 [Missing Semester: 代码质量](#)、[Missing Semester: 元编程](#)、[Missing Semester: 大杂烩](#)。

实例 1：用 Ruff 检查并修复嵌套判断。在 nested.py 中写入两层判断，只启用 SIM102 规则检查时，Ruff 指出两个 if 可以合并。普通的 --fix 不会应用这项修改，确认改动后加入 --unsafe-fixes，两个条件被合并到同一条判断中，复检通过，见图 12。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课后/quality (main)
$ ruff check --select SIM102 nested.py
SIM102 Use a single `if` statement instead of nested `if` statements
--> nested.py:2:5
1 | def classify(number):
2 | /   if number > 0:
3 | |   if number % 2 == 0:
4 | |       return "positive even"
5 | |   return "other"
help: Combine `if` statements using `and`

Found 1 error.
No fixes available (1 hidden fix can be enabled with the `--unsafe-fixes` option).

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课后/quality (main)
$ ruff check --select SIM102 --fix --unsafe-fixes nested.py
Found 1 error (1 fixed, 0 remaining).

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课后/quality (main)
$ ruff check --select SIM102 nested.py
All checks passed!

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课后/quality (main)
$ cat nested.py
def classify(number):
    if number > 0 and number % 2 == 0:
        return "positive even"
    return "other"
```

图 12 Ruff 发现并修复 SIM102 问题

实例 2：使用参数化测试覆盖多组输入。价格计算函数接收原价和折扣，使用 `pytest.mark.parametrize` 设置无折扣、八折和五折三组输入。指定该测试函数运行后，三组结果均通过，见图 13。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课后/quality (main)
$ ls price.py tests/test_price.py
python -m pytest -q tests/test_price.py::test_final_price
price.py tests/test_price.py
...
3 passed in 0.02s [100%]
```

图 13 参数化测试的三组输入全部通过

实例 3：添加并单独运行回归测试。针对负数价格补充回归测试，要求函数抛出带有 `non-negative` 提示的 `ValueError`。使用 `-k regression` 后，Pytest 从四项测试中只运行该项，得到一项通过、三项未选择，见图 14。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课后/quality (main)
$ python -m pytest -k regression -vv
===== test session starts =====
platform win32 -- Python 3.11.8, pytest-9.1.1, pluggy-1.6.0 -- D:\Software\python.exe
cachedir: .pytest_cache
rootdir: D:\desktop\系统开发工具基础\第四周\课后\quality
plugins: anyio-4.12.0, cov-7.1.0
collected 4 items / 3 deselected / 1 selected

tests/test_price.py::test_negative_price_regression PASSED

===== 1 passed, 3 deselected in 0.02s =====
```

图 14 使用关键词单独运行回归测试

实例 4：使用 Mock 模拟外部接口。用 Mock 代替接口客户端，将 `/rate` 的返回值设为 0.8，再检查函数结果和实际调用参数。该测试不访问网络，单独运行后通过，见图 15。

```

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课后/quality (main)
• $ python -m pytest -k without_network -vv
===== test session starts =====
platform win32 -- Python 3.11.8, pytest-9.1.1, pluggy-1.6.0 -- D:\Software\python.exe
cachedir: .pytest_cache
rootdir: D:\desktop\系统开发工具基础\第四周\课后\quality
plugins: anyio-4.12.0, cov-7.1.0
collected 5 items / 4 deselected / 1 selected

tests/test_price.py::test_get_rate_without_network PASSED [100%]

===== 1 passed, 4 deselected in 0.02s =====

```

图 15 使用 Mock 完成不访问网络的接口测试

实例 5：检查并提高测试覆盖率。用 `pytest-cov` 运行全部测试并生成终端与 HTML 报告。第一次运行时 `price.py` 第 5 行未覆盖，覆盖率为 88%；加入折扣超出范围的测试后，六项测试全部通过，遗漏行数变为 0，覆盖率达到 100%，见图 16。

```

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课后/quality (main)
• $ pytest --cov=price --cov-report=term-missing --cov-report=html -q
===== tests coverage =====
coverage: platform win32, python 3.11.8-final-0
Name      Stmts  Miss  Cover   Missing
-----
price.py    8      1    88%      5
TOTAL      8      1    88%
Coverage HTML written to dir htmlcov
5 passed in 0.23s

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课后/quality (main)
• $ pytest --cov=price --cov-report=term-missing --cov-report=html -q
===== tests coverage =====
coverage: platform win32, python 3.11.8-final-0
Name      Stmts  Miss  Cover   Missing
-----
price.py    8      0   100%
TOTAL      8      0   100%
Coverage HTML written to dir htmlcov
6 passed in 0.14s

```

图 16 补充异常分支测试前后的覆盖率对比

实例 6：用 `grep` 查找危险的 `shell=True`。在示例文件中写入三种不同格式的 `subprocess.Popen` 调用。第一个表达式只能匹配参数紧密书写的情况；加入空白字符规则后可以匹配前两种写法，但仍无法跨行处理第三种写法。

```

1 grep -En 'subprocess\.Popen\(.*shell=True' unsafe.py
2 grep -En 'subprocess\.Popen\(.*shell[[:space:]]*=[[:space:]]*True' unsafe.py

```

代码 6 使用两种正则查找 `shell=True`

两次查找分别得到一行和两行结果，见图 17。字符级正则可以处理简单搜索，但不适合直接分析跨行代码结构。

```

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课后/regex (main)
• $ grep -En 'subprocess\.Popen\(.*shell=True' unsafe.py
3:subprocess.Popen("dir", shell=True)

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课后/regex (main)
• $ grep -En 'subprocess\.Popen\(.*shell[[:space:]]*=[[:space:]]*True' unsafe.py
3:subprocess.Popen("dir", shell=True)
4:subprocess.Popen("dir", cwd=".", shell = True)

```

图 17 两种 `grep` 正则对不同代码格式的匹配结果

实例 7：用捕获组提取日期中的月份。使用 `findall` 搜索 2026-09-14 和 2026-10-01，表达式中的括号只捕获月份，得到 `['09', '10']`，见图 18。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课后/regex (main)
$ python -c 'import re; text="2026-09-14 and 2026-10-01"; print(re.findall(r"\d{4}-(\d{2})-(\d{2})", text))'
['09', '10']
```

图 18 正则捕获组提取出的两个日期月份

实例 8：比较贪婪与改进后的 JSON 正则。示例字符串的姓名中包含转义双引号。直接使用 `.*` 会一直匹配到 JSON 中最后一个引号，改进后的表达式则把转义字符和普通非引号字符分别处理。

```
1 import re
2
3 text = r'{"name": "Alyssa \"P.\" Hacker", "college": "MIT"}'
4
5 bad = re.search(r'"name"\s*:\s*"(.*)"', text).group(1)
6 good = re.search(r'"name"\s*:\s*"((?:\\.|[^\\"\\])*)"', text).group(1)
7
8 print("greedy:", bad)
9 print("fixed: ", good)
```

代码 7 处理转义引号的姓名捕获代码

图 19 中，贪婪写法错误地包含了 `college` 字段，修改后的结果只保留姓名内容。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课后/regex (main)
$ python extract_name.py
greedy: Alyssa \"P.\" Hacker\", \"college\": \"MIT\"
fixed: Alyssa \"P.\" Hacker
```

图 19 贪婪正则与改进后正则的捕获结果

实例 9：使用 JSON 解析器读取标准输入。将同一段 JSON 通过管道交给 Python，`json.load` 从标准输入读取对象后访问 `name` 字段，姓名中的双引号被正确还原，见图 20。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第四周/课后/regex (main)
$ printf '%s\n' '{"name": "Alyssa \"P.\" Hacker", "college": "MIT"}' |
python -c 'import json,sys; print(json.load(sys.stdin)["name"])'
Alyssa "P." Hacker
```

图 20 使用 JSON 解析器得到的姓名字段

实例 10：比较不同的版本约束。使用 Packaging 的 `SpecifierSet` 对五个版本应用兼容版本、通配符和范围三种约束。

```
1 from packaging.specifiers import SpecifierSet
2
```

```
3 versions = ["1.2.9", "1.3.0", "1.3.8", "1.9.0", "2.0.0"]
4
5 for rule in ["~=1.3", "==1.3.*", ">=1.3,<2"]:
6     matched = [v for v in versions if v in SpecifierSet(rule)]
7     print(f"{rule:10} -> {matched}")
```

代码 8 判断版本是否满足三种约束

如图 21 所示, `==1.3.*` 只接受 1.3 系列, 而 `~=1.3` 和 `>=1.3,<2` 均接受 1.3 以上、2.0 以下的版本。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课后/meta (main)
$ python semver_check.py
~=1.3      -> ['1.3.0', '1.3.8', '1.9.0']
==1.3.*    -> ['1.3.0', '1.3.8']
>=1.3,<2   -> ['1.3.0', '1.3.8', '1.9.0']
```

图 21 三种版本约束对应的匹配结果

实例 11: 生成并验证精确依赖锁定文件。从 `pip list --format=freeze` 的结果中筛选 Packaging、Pytest、pytest-cov 和 Ruff, 按名称排序后写入 `requirements.lock`。随后使用 `--dry-run` 检查锁定文件, 四个指定版本均已安装, 见图 22。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课后/meta (main)
$ python -m pip list --format=freeze |
grep -Ei '^(packaging|pytest|pytest-cov|ruff)=' |
sort > requirements.lock

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第四周/课后/meta (main)
$ cat requirements.lock
python -m pip install --dry-run -r requirements.lock
packaging==25.0
pytest==9.1.1
pytest-cov==7.1.0
ruff==0.16.5
Requirement already satisfied: packaging==25.0 in D:\Software\Lib\site-packages (from -r requirements.lock (line 1)) (25.0)
Requirement already satisfied: pytest==9.1.1 in D:\Software\Lib\site-packages (from -r requirements.lock (line 2)) (9.1.1)
Requirement already satisfied: pytest-cov==7.1.0 in D:\Software\Lib\site-packages (from -r requirements.lock (line 3)) (7.1.0)
Requirement already satisfied: ruff==0.16.5 in D:\Software\Lib\site-packages (from -r requirements.lock (line 4)) (0.16.5)
Requirement already satisfied: colorama>=0.4 in C:\Users\xya\AppData\Roaming\Python\Python311\site-packages (from pytest==9.1.1->-r requirements.lock (line 2)) (0.4.6)
Requirement already satisfied: iniconfig>=1.0.1 in D:\Software\Lib\site-packages (from pytest==9.1.1->-r requirements.lock (line 2)) (2.3.0)
Requirement already satisfied: pluggy<2,>=1.5 in D:\Software\Lib\site-packages (from pytest==9.1.1->-r requirements.lock (line 2)) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in D:\Software\Lib\site-packages (from pytest==9.1.1->-r requirements.lock (line 2)) (2.19.2)
Requirement already satisfied: coverage>=7.10.6 in D:\Software\Lib\site-packages (from coverage[toml]>=7.10.6->pytest-cov==7.1.0->-r requirements.lock (line 3)) (7.16.0)

[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

图 22 生成依赖锁文件并检查锁定版本

3 解题感悟

最后一周做下来, 感觉这几道题比前面更像是在把之前学过的东西连起来。q13 做完明白 Ruff 和 Pytest 不是一回事, Ruff 检查格式, 而 Pytest 检查运行结果。把几条命令放进 `check.sh` 后可以省事不少, 之后改完代码不用再回忆要检查什么, 直接跑一次脚本就行。

q14 让我更好地理解了 Make 的依赖关系。之前 C++ 的课程作业中用过类似的构建工具 CMake, 但基本都是照着已有的 `CMakeLists.txt` 修改, 没有认真想过构建工具是怎么判断哪些文件需要重新生成的。这次先完成第一次构建, 再观察无改动时的结果,

最后用 `touch data.csv` 触发重新构建，才比较直观地看懂依赖和文件修改时间的作用。Makefile 也不只是给长命令起几个名字，依赖写得准确才能避免重复执行，也不会漏掉真正需要更新的文件。

q15 实际做的时候反而是环境最麻烦，电脑里一开始没有能直接调用的 jq，还要先把工具路径处理好。不过装好之后，用 curl 和 jq 串起来筛选 JSON 确实很方便，数据不复杂时，一条管道就能把筛选、排序和输出格式一起完成。

最后的 q16 算是把测试、质量检查、构建和 Git 提交重新走了一遍。故意把问候语改错时，Ruff 仍然通过，但测试马上报错，这一点让我印象比较深：代码没有格式问题不等于功能正确。回头看这四周，从 Shell、Git、LaTeX 到调试、打包和自动检查，单个命令我之后可能还是会忘，但遇到问题时应该先看什么、怎么留下可验证的结果，比刚开始上课时清楚了不少。

第四周课上实验、课后练习及实验报告均已提交至 Git 仓库，相关提交记录见图 23。

```
D:\desktop\系统开发工具基础\第四周>git log --oneline -8
d2b1ea5 (HEAD -> main, origin/main, origin/HEAD) 第四周实验报告初版完成
c7ee741 第四周完成课后练习
47c5dd8 补充q16剩余过程
99480f6 完成 q16 综合质量检查与构建流程
831c721 第四周课上q15任务完成
b94d2b2 第四周课上q14任务完成
d29df60 删除了q13中复制过来的无用东西
648478c 第四周课上q13任务完成
```

图 23 第四周实验及报告的 Git 提交记录